Selecting Reliable LLM Reasoning — MP2 Report

DASE7506 Advanced Machine Learning Methods · Fall 2026

Final results: validation 93.48% (746/798) · test 81.80% (382/467) · inference assets 13.8 KB · full CPU evaluation ≈ 0.3 s

---

1. Task and summary

Given a GSM8K-style maths question and four candidate solutions produced by Qwen3-0.6B, the task is to pick one candidate whose **final numeric answer** is correct. Every group contains at least one correct answer, and any correct pick earns the point, so the task is *recognition*, not *verification*.

My final method is a **linear group-softmax ranker**: each candidate is described by 23 hand-crafted features, their group-normalized copies, and the supplied 1024-dimensional trace embedding (1070 features in total). A linear model scores each candidate; training minimizes, per question group, the negative log probability mass that a softmax over the four scores assigns to the set of *correct* candidates — exactly the quantity the scorer rewards. At inference the argmax candidate is returned. Training fits on the train split only, is fully deterministic (zero-initialized full-batch Adam, no sampling), takes 23 s on CPU, and produces a 13.8 KB checkpoint.

2. What the data looks like

Split	Groups	P(candidate correct)	Groups with 4/3/2/1 correct
train	4,755	75.5%	2242 / 1070 / 732 / 711
validation	798	75.7%	379 / 179 / 123 / 117
test	467	**51.1%**	0 / 171 / 145 / 151

Train and validation are *easy*: in almost half of the groups all four candidates are correct, so any selector scores 100% on them. The test bank is explicitly harder — no all-correct groups, and at least two distinct plausible answers per group. Two consequences shaped the whole development protocol:

1. **Method selection must use the hard groups of validation.** Only the 419/798 validation groups with mixed correctness carry any signal; the overall validation accuracy is half noise-free. All model comparisons below therefore report `val-hard` (accuracy on mixed-correctness groups) alongside `val-all`.
2. **A distribution shift to expect.** Test is harder than even the hard validation groups (51% vs ~59% candidate correctness), so a drop from val-hard to test was expected; the observed 87.6% → 81.8% is consistent with that.

Simple baselines (validation, overall / hard-only):

Baseline	val-all	val-hard
First candidate	78.20	58.47
Random among parseable	80.33	62.53
Longest candidate	77.94	58.00
Majority vote on parsed answers	86.22	73.75

Majority voting is strong but far from sufficient on hard groups, where several wrong candidates often agree with each other. Note that *long* answers are slightly *worse* than chance — verbosity is not reliability.

3. Method

3.1 Features (per candidate, 1070 dimensions)

Hand-crafted (23): text statistics (log length, log digit count, log equation count, `\boxed` / `####` / "final answer" markers, line count); parse success of the official numeric parser; **answer agreement** — how many other candidates share the same parsed final answer, and whether all four agree; embedding geometry (norm, cosine to group centroid, mean and max cosine to the other three candidates); candidate position; hedging-word count ("wait", "actually", …); lexical overlap with the question; question length and number count; and answer-value features (log magnitude, is-integer, is-zero, appears-in-question).

Group-normalized copies (23): each feature minus its group mean, divided by its group standard deviation. These make features comparable across questions of different difficulty, which matters under the train→test difficulty shift.

Fixed embeddings (1024): the supplied whole-trace representations, used as-is.

3.2 The group-softmax objective

The scorer rewards picking *any* correct candidate. A natural training objective is therefore the probability that a softmax policy over the four scores picks a correct one. For a group with scores $s \in \mathbb{R}^4$ and correct set C:

$$\mathcal{L} = -\log \sum_{j \in C} \mathrm{softmax}(s)_j, \qquad s_j = w^\top x_j + b$$

This differs from pointwise binary logistic regression ("is this candidate correct?") in a crucial way: **all-correct groups contribute exactly zero loss** (the sum is 1), so training automatically concentrates on the hard, mixed groups — the test-like cases — without any manual subsetting. It also handles groups with 2–3 correct candidates gracefully, rewarding mass on *any* of them rather than forcing a single target.

The model is linear, trained with full-batch Adam (1500 epochs, lr 0.02, L2 λ = 0.02) from a zero initialization. Because there is no mini-batch sampling, **training is bit-for-bit deterministic** — the seed-variance problems of stochastic training (Section 4.2) disappear by construction.

3.3 Inference

Standardize features with train statistics, apply the linear model, take the per-group argmax. Numpy only; no reference answers are read; nothing about question counts or IDs is assumed, so the same frozen `predict()` runs unchanged on the audit bank.

4. Experiments

4.1 Model comparison (train split fit, validation selection)

Method	val-all	val-hard
Majority vote	86.22	73.75
Logistic regression, hand features only	89.85	80.67
Logistic regression, embeddings only	88.97	79.00
Logistic regression, hand + embeddings (pointwise)	91.60	84.01
Gradient boosting (tuned), hand features	88.97	79.00
MLP-64 (pointwise, early stopping)	91.48	83.77
Group-softmax linear, hand only	90.23	81.38
Group-softmax linear, embeddings only	90.35	81.62
Group-softmax linear, hand + embeddings (submitted)	**93.48**	**87.59**
Group-softmax + PCA-128 embeddings	92.48	85.68
Group-softmax tiny MLP (h=32, λ=0.03)	93.36	87.35
Group-softmax + TF-IDF (2k–8k terms, $\lambda \le 0.05$)	≤91.60	≤84.01

Findings:

- **The objective matters more than the model class.** Pointwise logistic regression on identical features loses 3.6 points on val-hard to the group-softmax linear model (84.0 → 87.6). A tiny

MLP trained with the same group-softmax loss adds nothing over the linear version (87.35 vs 87.59) while costing 35× the parameters.

- **Hand features and embeddings are complementary.** Alone they reach 78.8–81.6 val-hard; together 87.6. The embeddings carry semantic signals the regexes cannot see (and vice versa: the parser/agreement features are invisible to a 0.6B embedding).
- **Nonlinear models overfit.** Gradient boosting reaches 93.5–97.7% on train but ≤79.0 on val-hard; the pointwise MLP similarly gaps (90.1 train-hard vs 83.8 val-hard). With 19k samples and 1070 features, capacity is the enemy.
- **Internal-consistency features** (does the final answer match the last "= …" in the trace; do `\boxed` and `####` conflict) *hurt* slightly (86.6 vs 87.6) and were removed — a failed idea worth reporting: wrong traces usually *are* internally consistent; they just solve the wrong problem.
- **Selection rule:** pooling probability mass over identical answers ("cluster") was never better than plain argmax and was dropped.
- **TF-IDF text features are pure overfitting here.** Adding a 2,000–8,000-term unigram+bigram TF-IDF block (vocabulary and IDF learned from train candidate texts only) drives train-hard to 99–100% while *lowering* val-hard by 3.5–5.5 points: the model memorizes question-specific lexical patterns that do not transfer. The supplied embeddings already cover the transferable semantics.
- **Five further ideas, five null results.** (i) "Independent agreement" features (same answer but *dissimilar* trace, to discount paraphrase consensus): val-hard 87.11 vs 87.59 baseline — no gain, so paraphrase agreement appears just as informative as independent agreement for this generator. (ii) Block-wise L2 (weaker penalty on hand features, stronger on embeddings): identical 87.59 at best. (iii) Up-weighting single-correct groups 1.5–2×: monotonically worse (86.16 / 84.73) — the group-softmax loss already concentrates on hard groups, and extra emphasis only distorts calibration. (iv) **Arithmetic verification** — actually evaluating every "expr = value" claim in the trace (79.5% of candidates contain at least one checkable computation): a detected arithmetic error lowers P(correct) only from 75.8% to 73.7%, and the features leave val-hard at ≤87.11. Wrong candidates from this generator mostly fail at problem *setup* (wrong operation, missed condition), not at arithmetic — echoing the bus example in the guide. (v) Ensembling the group-softmax with a pointwise logistic model (probability averaging): 87.59 at best, no gain. All changes were kept out of the final model; every configuration within ±0.5 points of 87.6 was treated as noise.

4.2 Why deterministic training

Mini-batch training of the same objective showed large seed variance (val-hard 81.9–87.6 across five seeds; 5-seed weight/probability ensembles landed at 86.4). Selecting the lucky seed would be tuning on validation noise. Switching to full-batch gradients from a zero init removes all randomness; across a 9-point (λ, lr, epochs) grid the result is a flat plateau at val-hard 86.4–87.6, and the submitted setting (λ=0.02, lr=0.02, 1500 epochs) sits mid-plateau rather than at a lucky spike.

4.3 Frozen test result

The method was frozen after Section 4.2 and run on test exactly once via the official pipeline:

	accuracy
Validation (official scorer)	**93.48%** (746/798)
Test (official scorer)	**81.80%** (382/467)
Majority-vote baseline on test	69.16%

The method improves over majority voting by **+12.6 points** on test. The gap between val-hard (87.6) and test (81.8) reflects the harder test mix (Section 2), not overfitting to validation: train-hard accuracy (87.8) ≈ val-hard (87.6), i.e. the model barely overfits its own training data.

5. Analysis

5.1 What the model actually uses

Largest hand-feature weights (standardized): `is_integer` (+0.32/+0.39 for the raw/z-scored versions), `parsed_ok` (+0.15/+0.18), `log_digits` (+0.16 z), `is_zero` (−0.10/−0.12), `agree` (+0.09 z), `log_eq` (+0.09 z), `log_len` (−0.08 z). Three observations:

- **An unparseable answer is nearly fatal** — the scorer cannot reward what it cannot read, and the model has learned that formatting *is* reliability.
- **"Round" integer answers of moderate magnitude are favored; zero is penalized.** Wrong traces more often produce degenerate or copied-from-the-question values.
- **Agreement helps but is deliberately cheap.** The model overrides the majority surprisingly often, and usually wins when it does (Section 5.3) — evidence it reads reasoning quality signals beyond counting votes.

5.2 Can the model recognise its own uncertainty?

On hard validation groups, accuracy by confidence bucket (softmax probability of the pick): 80.0% ($p<0.4$) ⟶ 89.0% ⟶ 85.8% ⟶ 90.6% ⟶ 94.4% ($p\geq0.85$); by top-2 margin: 84.9% (margin<0.05) ⟶ … ⟶ 90.4% (margin≥0.3). The trend is real but weak (Spearman $\rho \approx 0.07$): confidence says *something*, but the model cannot reliably flag its own failures. A selective-prediction system built on this confidence would barely beat the base rate — honest negative result.

5.3 Should candidates be judged jointly?

Yes. On hard validation groups the model picked a *minority* answer (against the most frequent one) 46 times and was right 35 of them (76%) — far above the ~51% base rate of such picks. Comparing candidates exposes missed conditions that per-candidate plausibility misses. Example: for the "3 naps a week, 2 hours each, 70 days" question, two candidates answer 60 (correct: 10 weeks × 6 h), two answer 420 (forgot the week conversion). The model picks 420 with $p=0.51$ — its one characteristic failure mode is precisely the *2-vs-2 split*, where agreement gives no signal and it must judge the reasoning itself.

5.4 Failure modes

Of the 52 hard-group failures on validation, the dominant patterns are: (i) 2-vs-2 answer splits where the wrong side is argued slightly more fluently; (ii) all four candidates wrong-sounding

with the single correct one phrased unusually (unparseable-ish formatting, atypical value); (iii) questions whose correct answer "looks wrong" to the learned priors (e.g. large non-integer results). The first failure in 5.3 is type (i); the Megan-eggs failure (picked 6 over two agreeing 9s) shows the model's integer/magnitude priors being exploited by a plausible miscount ("twice as many cracked" misread as a subtraction).

6. Resources and compliance

Quantity	Value	Limit
Inference assets (`runs/model_bundle.npz`: w, b, mu, sd as float32)	**13,806 bytes ≈ 0.013 MiB**	16 MiB
Test prediction wall time (incl. startup, imports, data load)	0.23 s	30 s total
Test scoring wall time	0.05 s	(same 30 s budget)
Training time (CPU, full pipeline from raw data)	23 s	—
Threads	4 (`OMP_NUM_THREADS=4` etc.)	4

Local hardware: Apple-silicon MacBook (arm64), Python 3.12.13, numpy 2.5.3. Inference is a single 1868×1070 matmul plus regex feature extraction; on the reference Xeon it remains orders of magnitude inside the budget. The method never opens reference files at inference, uses no external data or pretrained weights, and accepts arbitrary bank sizes.

7. Reproduction

```
cd code
python -m venv .venv && source .venv/bin/activate   # Python 3.12
python -m pip install -r requirements.txt            # numpy only
python check_data.py
python train_final.py                                # writes runs/
model_bundle.npz (23 s, deterministic)
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method submission.py --checkpoint runs/model_bundle.npz \
  --split test --output runs/test-predictions.json
python score_predictions.py --split test \
  --predictions runs/test-predictions.json --output runs/test-score.json
# expected: accuracy 81.7987...
```

The committed `runs/model_bundle.npz` lets a peer reproduce the score **without retraining**; `train_final.py` regenerates it bit-for-bit. Development scripts and the full experiment log are in `EXPERIMENTS.md`; the evaluation JSONs are in `runs/`.

8. AI assistance disclosure

The project was developed with AI coding assistance (Claude, Cherry Studio agent): the assistant proposed the feature set, implemented the training/evaluation scripts, ran the experiments, and drafted this report. All design decisions (group-softmax objective, deterministic full-batch training, hard-group selection protocol, feature ablations) were verified experimentally by the assistant and reviewed by the author, who takes responsibility for the final method and claims. No external code was copied; the only dependencies are numpy (final pipeline) and scikit-learn/scipy (development comparisons).

9. Conclusion

Aligning the training objective with the scoring rule — maximizing the softmax mass on the *set* of correct candidates — beat every pointwise alternative, and doing it with a deliberately small linear model beat every higher-capacity alternative. The most reliable signals that an answer is right are boring ones: it parses, it is a sensible integer, its digits were actually computed, and other candidates independently agree. The weakest point of the method is self-knowledge: its confidence is only weakly informative, and 2-vs-2 answer splits remain essentially coin flips weighted by style.